National College of Ireland

Project Submission Sheet

Student Name:	…Sabhyata Kumari……		
Student ID:	…X24283142 ……		
Programme:	Msc in Artificial Intelligence ……	**Year:**	2025/2026 ……
Module:	… Machine Learning (MSCAI1)……		
Lecturer:	20th March, 2026 ……		
Submission Due Date:	……		
Project Title:	Intelligent Proactive Resource Forecasting for Cloud-Native Systems: A Multi-Model Machine Learning Approach to SLA Prediction and Cross-Cloud Cost Optimisation……		
Word Count:	……		

Academic Integrity Certificate Verification Code: … 5G7NgSCwQV ……

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
<u>ALL</u> internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting **Monday, 19th January 2026**, **all students are required to successfully complete the Epigeum Academic Integrity Course**, which is available on **Moodle**. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: 🔗 ttps://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Signature:	Sabhyata Kumari ……
Date:	20th March, 2026 ……

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**

5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
[Insert Description of use]	
[Insert Sample prompt]	[Insert Sample response]

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

[Place evidence here]

NCI
Coláiste Náisiúnta na hÉireann | National College of Ireland

This certificate is awarded to:

Sabhyata Kumari

FOR COMPLETING THE PROGRAMME

Academic Integrity: For Students, Second Edition

December 19, 2025

Certificate Verification Code: 5G7NgSCwQV

epigeum

Grok

Intelligent Proactive Resource Forecasting for Cloud-Native Systems: A Multi-Model Machine Learning Approach to SLA Prediction and Cross-Cloud Cost Optimisation

I. MOTIVATION AND INTRODUCTION

Cloud infrastructure auto-scaling tools such as Kubernetes Horizontal Pod Autoscaler (HPA) operates reactively and it trigger resource provisioning only after the occurrence of breaches in CPU and memory thresholds [1]. During this 2–5 minute latency between load spikes and resource readiness, users suffer performannce degradation and possible violation of Service Level Agreements (SLA). Site Reliability Engineers (SREs) use over-provisioning (typically 30–40% above peak) to mitigate provisioning lag and handle unexpected traffic surges, ensuring high availability. According to Gartner estimates, global cloud waste due to over-provisioning and suboptimal auto-scaling crossed $26 billion in 2023 alone [1].

Reactive, threshold-based scaling creates structural failure modes by acting on past metrics rather than future demand. This is the architectural constraints/limitations. Threshold-based monitoring has three architectural failure modes in its structure. First, Detection Lag creates a temporal gap exists between when demand arrives and when the system detects it and triggers a scaling event.Second, the configuration of the conservative thresholds always requires permanent over-provisioning of the infrastructure to avoid risk.Third, Rapidly toggling between upscaling and downscaling when demand hovers near a threshold, resulting in instability.

Machine learning provides a different approach through proactive forecasting. Rather than reacting to crossed thresholds, ML models forecast resource utilisation 30–60 minutes in ahead, which allows systems to early scale before the actual demand spikes. Recent research in AIOps using ML for anomaly detection has shown performance incidents are detectable with ML-based anomaly detection up to 80% better than traditional threshold-based detection approaches [1]. However, existing AIOps research focuses primarily on post-incident diagnostics[5] or single-cloud anomaly detection[2]. This research addresses critical, under-explored gaps in AIOps: calculating the financial difference between proactive vs. reactive scaling and the cross-cloud portability of resource forecasting models trained on one provider's workload data.

This research aims to bridge this gap by using a systematic evaluation of three ML models: XGBoost, Random Forest, and LSTM for CPU and memory resource utilization forecasting at a 30-minute forecast horizon. The models will be trained using the Alibaba 2018 Cluster Trace dataset and then tested on Azure and Google Cloud data to assess cross-cloud portability. SHAP(SHapley Additive exPlanations) is used for SHAP-based feature attribution aligned to SRE decision contexts. A cost analysis using AWS EU West-1 region pricing is used to identify the financial impact of proactive ML-based scaling and reactive threshold-based scaling.

II. RESEARCH QUESTIONS

RQ1: Which ML model (XGBoost, Random Forest, or LSTM) most accurately predicts cloud CPU and memory utilisation 30 minutes ahead while maintaining the optimal balance between prediction accuracy and inference latency for real-time SRE?
RQ2: How much cloud infrastructure cost is saved by proactive ML-driven scaling compared to reactive threshold-based auto-scaling across different workload types?.
RQ3: Do resource forecasting models trained on one cloud provider's cluster data (Alibaba) generalise effectively to others (Azure, Google)without retraining?

III. INITIAL LITERATURE REVIEW

The intersection of machine learning, AIOps, and cloud performance engineering has attracted significant research attention in recent years. However, there are many critical gaps in existing literature that motivated this investigation.

Amgothu et al. [1] present a comprehensive survey on observability and AIOps for cloud-scale DevOps environments, documenting that ML based monitoring systems achieve a reduction in anomaly detection times of more than 80% and a reduction in false positive rates by 50% percent when compared to a threshold-based approach. Although the authors demonstrate very compelling performance improvements for a number of production environments and they only focus on anomaly detection.

Furthermore, no cross-cloud generalisa- tion evaluation is presented, and quantified cost savings are not reported.

Sami et al. [2] provide an extensive overview of explainable AIOps for trusted AI in cloud-scale DevOps automation and evaluate the existing XAI integration methods such as SHAP, LIME, and attention mechanisms. However, The authors identify a significant gap: most AIOps frameworks prioritise prediction accuracy over interpretability, reducing engineer trust in automated recommendations. Their work motivates this project's incorporation of SHAP explanations but focuses on anomaly classification rather than continuous resource utilisation forecasting.

Kurian John [3] is focused specifically on the use of AI for the performance engineering domain. The use of the ML models is to be explained to create trust among the practitioners. Moreover, the performance modeling is to take into consideration the system-level interdependencies. Based on the critical identification of the paper, the gap identified is the fact that the majority of the prediction models are only validated against a single benchmark dataset without cross-environment evaluation. This is the gap that directly motivating this project's cross-cloud experimental design.

Litoiu et al. [4] discuss the concept of self-adapting software systems using feedback loops for the continuous optimisation of performance engineering. The control-theoretic architecture presented to ensure the consistent fulfillment of SLAs in adaptive cloud deployment systems. However, the approach is reactive, it responds only after when changes in performance are observed, rather than predecting future resource needs. It also doesn't take into account the cost differences between different scaling strategies.

Gan et al. [5] have proposed SAGE, a machine learning based system for performance debugging in microservice systems. It shows high accuracy in identifying root causes when tested on the DeathStarBench benchmark. SAGE is a diagnostic tool. It identifies what went wrong once performance incident occurred rather than predicting issues in advance. This highlights the gap between reactive diagnosis and proactive forecasting, which this project aims to address.

Williams et al. [6] discuss the challenges of safe agentic AI in performance engineering, where they highlight that human in loop is necessary in autonomous scaling decisions. They have shown that unsupervised ML based scaling suggest without clear explanations then engineers tend to loss of trust in the decision-making processes which in turn leads to override actions that defeat the benefits of automation. This study's focus on both prediction accuracy and SHAP interpretability can be seen to have been supported by Williams et al. These findings support this project's both improving prediction accuracy and SHAP based interpretability.

These studies highlights three key gaps that this project aims to address: (i) a lack of a study on systematic cross-cloud model generalisation in resource forecasting. (ii) a lack of quantification of cost savings between proactive and reactive scaling. (iii) a lack of SHAP-based explainability in cloud resource forecasting in a multi-model comparative framework.

IV. DATA SOURCES AND STATISTICS

Three publicly available cloud cluster data sets are selected to enable cross-cloud generalisation evaluation:

Dataset	Source	Size	Key Features
Alibaba 2018 Cluster Trace	github.com/alibaba/clusterdata	4 GB (sample: 500K rows)	CPU%, mem%, job queue, task events
Azure Public Dataset	github.com/Azure/AzurePublic-Dataset	300 MB (manageable subset)	VM CPU, memory, disk I/O, network
Google Cluster Traces v3	research.google/tools/datasets	10% sample 24 GB	CPU request, usage, memory, task events

Table I. Dataset overview

All three data sets contain time-series resource utilisation metrics recorded from production cloud environments. Alibaba 2018 covers 4,000 physical machines across eight days, Azure provides VM-level metrices from approximately 2,000 virtual machines across 30 days, Google Cluster Traces v3 covers approximately 125,000 machines across 29 days, of which a 10% sample will be used for computational process. Each data set will be split in 70% training, 15% validation, and 15% testing ratio. Feature engineering will include lag features (1h, 6h, 24h), rolling window statistics (mean, standard deviation, minimum, maximum over 5, 15, and 60-minute windows), time-of-day, day-of-week cyclic encodings, and rate-of-change features to capture gradient trends preceding resource spikes.

V. METHODS

A. XGBoost (ML Method 1)

XGBoost (Extreme Gradient Boosting) [5] is a gradient-boosted decision tree ensemble method selected as a primary ML baseline approach for its strong performance on structured tabular data and computational efficiency. Time series features will be transformed into a tabular format using a sliding window , enabling XGBoost to capture temporal patterns without relying on sequential model architectures. It has several advantages which includes native handling of missing values, built-in regularisation (L1 and L2), and direct compatibility with SHAP explanations. Hyperparameters (such as learning rate, max depth, n_estimators, subsample ratio) will be tuned via five-fold cross-validation using Bayesian optimisation.

B. Random Forest (ML Method 2)

Random Forest is used as a second traditional machine learning approach. It consists of an ensemble of decision trees trained independently using bagging and random feature selection. Its inherent variance reduction through ensemble averaging and natural feature importance scores ensures that Random Forest provides a robust XGBoost. The same set of features will be used to allow direct comparison to XGBoost. Its Hyperparameters which includes number of estimators, maximum depth and minimum samples split will be tuned via grid search. Random Forest's feature importances provide an additional interpretability signal alongside SHAP analysis.

C. LSTM Network (Deep Learning Method)

Long Short-Term Memory (LSTM) networks address the the limitations of traditional feedforward models when working with sequential data. They do this by maintaining internal memory states that can capture long-term temporal dependencies. The architecture comprises two stacked LSTM layers (64 units each) followed by a Dense output layer, trained on raw 60-minute lookback windows at 10-second resolution. To reduce overfitting, dropout regularisation (rate = 0.2) is applied between LSTM layers. The model is trained using the Adam optimiser with learning rate scheduling and early stopping based on validation loss. Unlike XGBoost and Random Forest, the LSTM model does not use any manually engineered features on the input data and can learn temporal patterns that are not included in the lag features.

D. SHAP Explainability Layer

SHAP (SHapley Additive exPlanations) will be used for post-training explanation of the XGBoost and Random Forest models using the TreeExplainer algorithm, which computes exact Shapley values effecientely for tree-based models. For LSTM, DeepExplainer will provide approximate SHAP values. The results are presented through a range of visualisations, including global feature importance bar charts, beeswarm plots that show how feature values influence predictions, and force plots for individual predictions. These visualisations help SRE engineers better understand model behaviour and the reasoning behind specific scaling decisions. All SHAP outputs will be integrated directly into the Streamlit application for easy access and interaction.

E. Cost Analysis Component

A cost estimation module will be used to calculate and compare the financial differential between proactive ML-driven scaling and reactive threshold-based auto-scaling. Based on predicted utilisation trends, the module identifies the earliest safe point to trigger scaling and calculates the exact number of nodes required to maintain SLA compliance. Cloud cost is computed using publicly available AWS EC2 on-demand pricing (m5.xlarge instance type as reference). Reactive scaling is modelled with a fixed 2-minute provisioning delay and 40% over-provisioning buffer. The final cost difference is expressed in euros per day, allowing for a clear and practical assessment of business value.

VI. EVALUATION METHODS

A. Prediction Accuracy

Prediction accuracy will be evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE) for each model on each datasets. These metrics are well suited for continuous resource utilisation targets and allow for direct comparison between models and datasets. RMSE penalises large prediction errors, which reflects the higher cost of under-predicting (leading to SLA breaches) compared to over-predicting (resulting in unnecessary scaling). Statistical significance of differences in RMSE between models will be assessed using a paired Wilcoxon signed-rank test (alpha = 0.05).

B. Computational Efficiency

In addition to accuracy, system performance will be evaluated by measuring training time (seconds/minutes), inference latency (milliseconds per prediction batch), and model size (MB) for each model. Inference latency is important for real-time SRE applications, as models must produce predictions quickly enough to support automated scaling decisions. For example, a model which requires 500 ms per prediction may be too slow for systems that demand instant responses. Learning curves will be plotted by training models on 10%, 25%, 50%, 75%, and 100% of training data to assess scalability and data efficiency across all three models.

C. Cross-Cloud Generalisation

Models trained on Alibaba 2018 dataset will be evaluated directly on on Azure and Google datasets without retraining and their performance will be measured using RMSE. Additionally, transfer learning experiments will be conducted by fine-tuning Alibaba-trained models on 10% of Azure/Google data to quantify adaptation cost. SHAP feature importance rankings will be compared across datasets to identify which resource features generalise across cloud providers.

D. Cost Efficiency

Cost savings (EUR/day) from proactive ML scaling versus reactive auto-scaling will be calculated across all three workload datasets and three forecast horizons (15, 30, 60 minutes). Results will be presented as mean savings and confidence intervals across test set time windows.

REFERENCES

[1] S. Amgothu, A. K. Vedantham, and S. P. Muthe Gowda, "Observability and AIOps in cloud-scale DevOps: Technologies, architectures, challenges, and future trends," in *Proc. IEEE Int. Conf. Cloud Comput.*, Boston, MA, USA, 2024, doi: 10.23919/FRUCT67853.2025.11239168.

[2] M. A. Sami, A. Rehman, Z. Ahmad, and N. Bano, "Explainable AIOps: A deep survey on trustworthy and transparent AI in cloud-scale DevOps automation," *Spectrum Eng. Manag. Sci.*, vol. 3, no. 7, 2025, doi: 10.5281/zenodo.15867960.

[3] L. K. John, "AI for performance engineering and performance engineering for AI," in *Proc. 16th ACM/SPEC Int. Conf. Perform. Eng. (ICPE)*, Toronto, ON, Canada, 2025, pp. 1–2, doi: 10.1145/3676151.3720528.

[4] M. Litoiu, "Closing the loop: Building self-adaptive software for continuous performance engineering," in *Proc. 15th ACM/SPEC Int. Conf. Perform. Eng. Companion (ICPE Companion)*, London, U.K., 2024, pp. 258–259, doi: 10.1145/3629527.3652910.

[5] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou, "Sage: Practical and scalable ML-driven performance debugging in microservices," in *Proc. 26th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, Virtual, USA, 2021, pp. 135–151, doi: 10.1145/3445814.3446700.

[6] D. Williams, M. Craun, M. V. Le, J. Stephen, S. Ahmed, and H. Jamjoom, "Towards safe agentic AI performance engineering," in *Proc. 4th Workshop Practical Adoption Challenges Mach. Learn. Syst. (PACML)*, Seoul, South Korea, 2025, pp. 63–68, doi: 10.1145/3766882.3767179.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[8] Alibaba Group, "Alibaba Cluster Trace 2018," [Online]. Available: https://github.com/alibaba/clusterdata. [Accessed: Mar. 20, 2026].

[9] Microsoft, "Azure Public Dataset," [Online]. Available: https://github.com/Azure/AzurePublicDataset. [Accessed: Mar. 20, 2026].

[10] Google, "Google Cluster Traces," [Online]. Available: https://research.google/tools/datasets/. [Accessed: Mar. 20, 2026].